

Disciplina de Inteligência Artificial , Professor Munif , Unicesumar 2026

Previsão de Churn de Clientes de Telecomunicações

Integrantes

- Miguel Drozino - RA: 23155078-2
- Micaela Dorneles - RA: 23150068-2
- Nathan Rodrigues - RA: 23025003-2
- Igor Costa - RA: 23215764-2

Resumo do Projeto

Contextualização

A retenção de clientes é um dos maiores desafios do setor de telecomunicações. O fenômeno chamado de churn ocorre quando um cliente encerra seu contrato com a operadora e migra para a concorrência. Identificar antecipadamente quais clientes têm maior probabilidade de cancelar permite ações preventivas como descontos e personalização de planos, reduzindo perdas financeiras.

Problema

Dado um conjunto de informações sobre o perfil e comportamento de um cliente de telecomunicações, é possível prever se ele irá cancelar o serviço (churn = Sim) ou permanecer (churn = Não)? Trata-se de um problema de classificação binária.

Hipótese

Clientes com contrato mensal, alto valor de cobrança mensal e pouco tempo de relacionamento com a empresa têm maior probabilidade de cancelar o serviço. Além disso, espera-se que o Random Forest apresente desempenho superior à Árvore de Decisão simples, especialmente nas métricas de F1-Score e Revocação.

Dataset

Item	Detalhe
Nome	Telco Customer Churn (simulado)
Origem	Baseado no dataset público do Kaggle (blastchar/telco-customer-churn)
Registros	7.043
Atributos	21 (20 preditores + 1 variável alvo)
Variável alvo	Churn (Yes / No)
Distribuição	~73,6% Não Churn ~26,4% Churn

Principais atributos

Atributo	Tipo	Descrição
tenure	Numérico	Meses de relacionamento com a empresa
MonthlyCharges	Numérico	Valor da cobrança mensal
TotalCharges	Numérico	Total cobrado ao longo do contrato
Contract	Categórico	Tipo de contrato (mensal, anual, bienal)
InternetService	Categórico	Tipo de serviço de internet
PaymentMethod	Categórico	Método de pagamento
SeniorCitizen	Binário	Cliente é idoso (1 = sim, 0 = não)

Tratamento dos dados

- Remoção da coluna customerID (identificador sem valor preditivo)
- Conversão de TotalCharges para numérico (valores ausentes preenchidos com a mediana)
- Codificação de variáveis categóricas com LabelEncoder (sklearn)
- Divisão treino/teste: 80% treino / 20% teste, com estratificação por classe
- Uso de class_weight='balanced' nos modelos para lidar com o desbalanceamento

Métodos de IA Utilizados

Parte 1 — Árvore de Decisão (DecisionTreeClassifier)

A Árvore de Decisão é um método supervisionado que constrói uma estrutura hierárquica de regras de decisão. O modelo aprende a dividir os dados em subconjuntos cada vez mais homogêneos com relação à variável alvo, por meio de critérios como Gini ou Entropia. É interpretável e permite visualização das regras aprendidas.

- max_depth=8 - profundidade máxima para evitar overfitting
- min_samples_leaf=20 - mínimo de amostras por folha
- class_weight='balanced' - peso proporcional inverso à frequência de classe

Parte 2 — Random Forest (RandomForestClassifier)

O Random Forest é um método de ensemble baseado em Bagging (Bootstrap Aggregating): treina múltiplas árvores de decisão em subconjuntos aleatórios dos dados e combina suas predições por votação majoritária. Por usar aleatoriedade também na seleção de atributos em cada split, reduz a variância e melhora a generalização.

- n_estimators=200 - 200 árvores na floresta
- max_depth=12 - profundidade máxima de cada árvore
- min_samples_leaf=5 - mínimo de amostras por folha
- class_weight='balanced' - balanceamento de classes

Avaliação dos Modelos

Métricas

Métrica	Árvore de Decisão (Parte 1)	Random Forest (Parte 2)
Acurácia	0.6380	0.6927
Precisão	0.3985	0.4372
Revocação	0.7358	0.5822
F1-Score	0.5170 **	0.4994

Gráfico 1 — Comparação de Métricas entre os Modelos

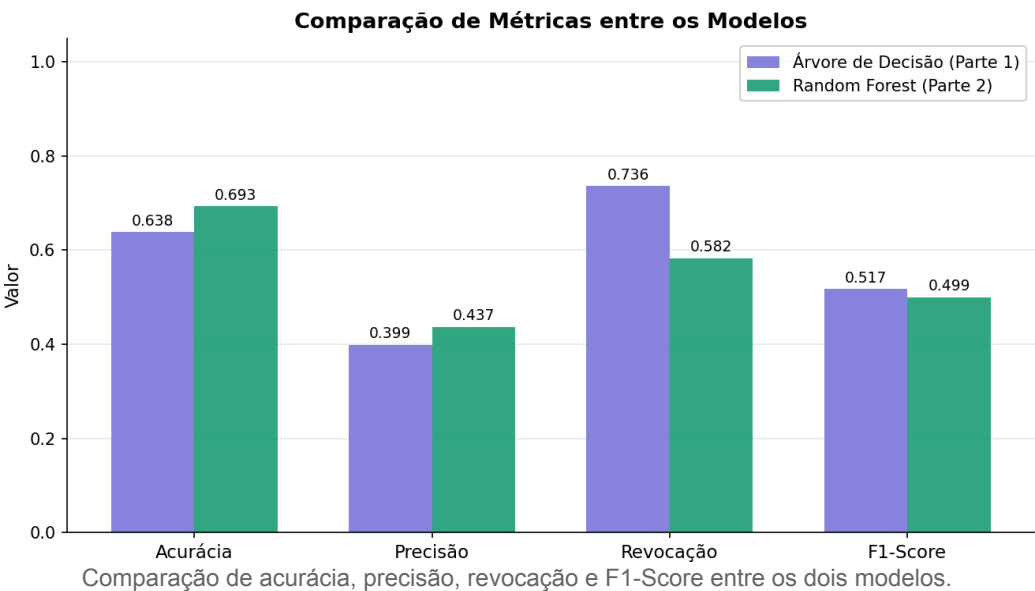
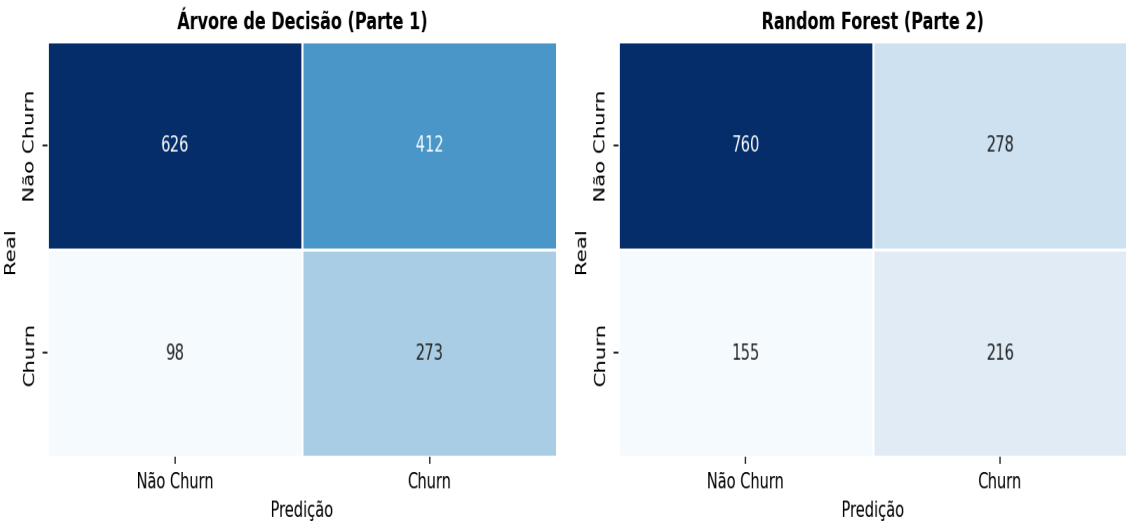


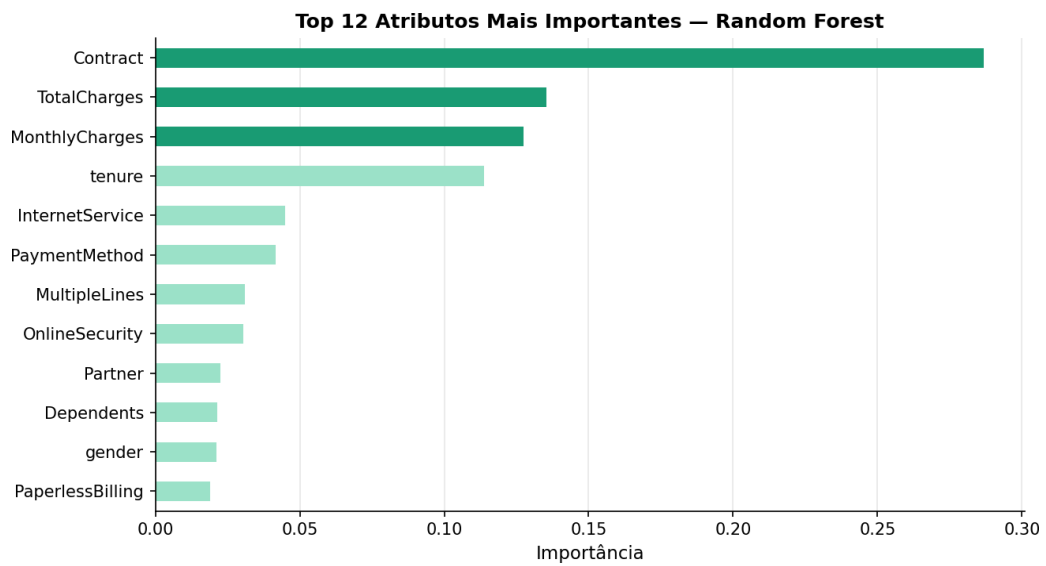
Gráfico 2 — Matrizes de Confusão

Matrizes de Confusão



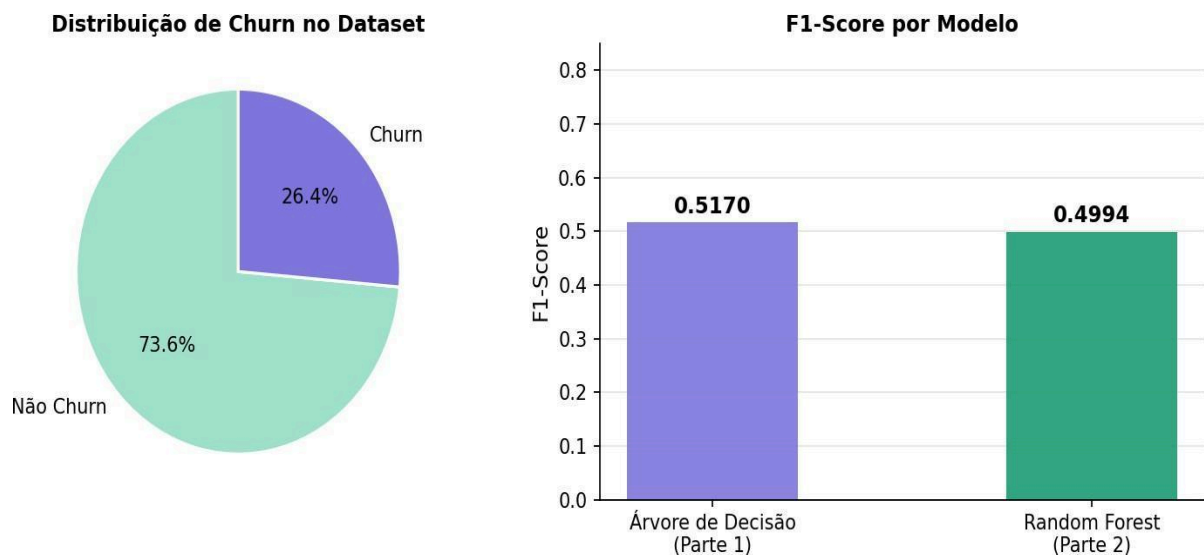
Distribuição de acertos e erros de cada modelo para as classes Churn e Não Churn.

Gráfico 3 — Importância de Atributos (Random Forest)



Os 12 atributos mais relevantes para a predição pelo modelo Random Forest.

Gráfico 4 — Distribuição de Churn e F1-Score por Modelo



À esquerda: proporção de churn no dataset. À direita: F1-Score comparativo.

Comparação dos Resultados

A Árvore de Decisão apresentou maior Revocação (0.7358) e melhor F1-Score (0.5170), tornando-a mais adequada para este problema, onde o custo de não identificar um cliente que vai cancelar (falso negativo) é alto. O Random Forest obteve maior Acurácia e Precisão, mas à custa de uma Revocação menor, ou seja, deixou de identificar mais clientes em risco de churn.

Em problemas de churn, a Revocação é frequentemente a métrica mais importante, pois o negócio prefere contatar um cliente que não cancelaria (falso positivo) a perder um cliente que de fato cancelaria (falso negativo). Nesse contexto, a Árvore de Decisão com balanceamento de classes se mostrou a melhor escolha.

Conclusão

Este trabalho demonstrou a aplicação prática de dois métodos de Inteligência Artificial para resolver um problema real de classificação. A Árvore de Decisão e o Random Forest foram treinados, avaliados e comparados de forma sistemática.

A hipótese foi parcialmente confirmada: os atributos tenure, MonthlyCharges, TotalCharges e Contract figuram entre os mais importantes. Porém, o Random Forest não superou a Árvore em F1-Score, o que pode ser explicado pelo desbalanceamento do dataset e pela configuração dos hiperparâmetros. O processo completo de desenvolvimento de uma solução baseada em IA foi exercitado: definição do problema, preparação dos dados, treinamento, avaliação com métricas e gráficos, comparação entre modelos e conclusão analítica.

Como trabalho futuro, poderíamos explorar técnicas de reamostragem, ajuste de hiperparâmetros por e outros métodos do segundo bimestre como AdaBoost ou Stacking.

Como Executar

1. Clone o repositório e entre na pasta do projeto.
2. Instale as dependências: `pip install pandas scikit-learn matplotlib seaborn joblib`
3. Execute o script principal: `python algoritmo-ia.py`

Os gráficos serão salvos em `graficos_resultados/` e os modelos treinados em `modelos_treinados/`. Para carregar um modelo salvo: `modelo = joblib.load('modelos_treinados/random_forest.pkl')`